Monte Carlo Methods for Particle Transport / Neutron Diffusion Parallel Stochastic Simulation with MPI and OpenMP

Course: Advanced Methods for Scientific Computing

Project Brief

Implement a basic Monte Carlo particle transport simulation in a simple 2D slab geometry with isotropic scattering and absorption. The aim is to estimate spatially resolved flux and reaction rates, and to parallelize across independent particle histories. Deliver a working code (C++ with possibly python scripts for visualization), validate on simple cases, quantify statistical errors, and study parallel performance.

1 Physical Model and Problem Setup

1.1 Geometry and Materials

We consider a homogeneous 2D slab domain $\Omega = [0, L_x] \times [0, L_y]$. The material is characterized by macroscopic cross sections:

$$\Sigma_t = \Sigma_s + \Sigma_a,$$

where Σ_t is the total cross section, Σ_s the scattering cross section, and Σ_a the absorption cross section. Optionally, one can include fission Σ_f for extended studies (not required for the baseline).

1.2 Source and Boundary Conditions

- External source: a monoenergetic, isotropic source distributed uniformly or localized at a boundary/region. - Boundary conditions: vacuum (particle escapes when it crosses the boundary), or reflective (specular reflection) for selected sides to enable simple validations.

1.3 Transport Assumptions

- Monoenergetic neutrons/particles. - Isotropic scattering in angle. - Path lengths sampled from an exponential distribution with mean free path $\lambda = 1/\Sigma_t$. - In each collision: with probability $p_s = \Sigma_s/\Sigma_t$ the particle scatters; with probability $p_a = \Sigma_a/\Sigma_t$ it is absorbed (history terminates).

2 Monte Carlo Algorithm

2.1 Core Loop

Each particle history proceeds as follows:

1. Initialize position $\mathbf{x}_0$, direction $\boldsymbol{\omega}_0$ uniformly on the unit circle; assign weight $w = 1$.
2. Sample a free-flight distance $\ell = -\ln(\xi)/\Sigma_t$, with $\xi \sim U(0, 1)$.
3. Move to
$$\mathbf{x} \leftarrow \mathbf{x} + \ell\boldsymbol{\omega}$$
. If boundary is crossed before distance ℓ, handle boundary condition (escape or reflection).
4. Tally track-length or collision estimators in the spatial bin(s) traversed.
5. Sample interaction: absorption with prob. p_a (terminate) or scattering with prob. p_s (draw new direction isotropically, continue).
6. Apply variance reduction (Russian roulette/splitting) as configured.

2.2 Estimators

Two common flux estimators:

- Track-length estimator (preferred for flux): accumulate$w \times$track-length$/V_{\text{bin}}$ per bin; yields lower variance for flux.
- Collision estimator: increment w/Σ_t at each collision in the current bin.

Reaction rate for a reaction Σ_r can be estimated as $\langle \Sigma_r \phi \rangle$ via track-length flux estimate times Σ_r per bin or via collision tallies specific to reaction events.

2.3 Statistical Analysis

Let $\hat{X}$ denote an estimator over N i.i.d. particle histories with samples X_i. The sample mean and unbiased variance are:

$$\bar{X} = \frac{1}{N}\sum_{i=1}^{N} X_i, \quad s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(X_i - \bar{X})^2.$$

Assuming approximate normality by the central limit theorem, a $100(1-\alpha)\%$ confidence interval is $\bar{X} \pm z_{\alpha/2}\frac{s}{\sqrt{N}}$. Report relative error

$$R = \frac{s/\sqrt{N}}{|\bar{X}|}$$

and the figure of merit (FOM) $\text{FOM} = \frac{1}{R^2 T}$, where T is wall-clock time.

3 Random Number Generation and Reproducibility

3.1 RNG Choices

Use the C++ random numbers or more specialised library for random number generator, like www.pcg-random.org, to generate random numbers. You may use `std::seed_seq` or similar tools to generate different seed across threads/ranks and avoid correlated streams.

3.2 Seeding Strategy

- Global base seed s_0.
- MPI rank r and OpenMP thread t get unique subseeds, e.g. using `std::seed_seq`.
- Ensure deterministic mapping: same particle count and layout yields identical results across runs if using same seeds.

4 Variance Reduction

4.1 Russian Roulette and Splitting

- Russian roulette: if weight $w < w_{\text{cut}}$, kill with probability p_k or boost weight to w/p_s with prob. $p_s = 1 - p_k$ to keep unbiasedness. - Splitting: when entering important regions, split a particle into n copies with weight w/n.

4.2 Importance Sampling (Stretch Goal)

Introduce a spatial importance function $I(\mathbf{x})$ to guide splitting/roulette or bias source/directions, with weight corrections to maintain unbiasedness.

5 Tally Accumulation

5.1 Spatial Binning

Discretize the slab into $N_x \times N_y$ bins. Maintain per-bin accumulators for:

- Flux estimate (track-length or collision-based).
- Reaction rates (e.g., absorption $\Sigma_a \phi$, scattering $\Sigma_s \phi$).

Use double precision accumulators. For OpenMP, use per-thread private tallies to avoid false sharing and then reduce. For MPI, reduce per-rank tallies with `MPI_Reduce`/`MPI_Allreduce`.

6 Parallelization Scope

6.1 MPI Parallelization

- Distribute the total number of particle histories equally across ranks.
- Each rank uses its own RNG subsequence.
- Periodically or at the end, perform global reductions of tallies and sample moments via `MPI_Allreduce`.

6.2 OpenMP Parallelization

- Parallelize the loop over particle histories with `#pragma omp parallel for`.
- Use per-thread tallies (arrays) and combine with a manual reduction at the end to avoid contention.
- Alternatively, use reduction over user-defined types if supported efficiently.

6.3 Hybrid MPI+OpenMP (Stretch Goal)

Combine both levels: distribute batches across ranks; within each rank, parallelize histories with OpenMP. Ensure unique RNG streams across all (r, t) pairs.

7 Suggested Milestones

1. Implement physical model and RNG; validate with simple cases:
   - Pure absorption: analytic flux decay $\phi(x) \propto e^{-\Sigma_a x}$ in 1D surrogate.
   - No absorption (only scattering) with vacuum boundaries: check leakage probability vs. mean free path heuristics.
2. Add variance reduction (Russian roulette, splitting) and statistical error estimation; report confidence intervals and FOM.
3. Parallelize with MPI/OpenMP; ensure reproducibility with seeded per-rank and per-thread streams; validate that tallies match serial within statistics.
4. Performance study: strong scaling with fixed number of particles; weak scaling by increasing particles per rank; plot speedup vs. cores/ranks and FOM vs. threads.

8 Deliverables

- Source code with build instructions (CMake or equivalent).
- Plots of statistical error vs. number of particles; speedup and parallel efficiency plots.
- A short report (4–6 pages) summarizing model, algorithms, validation, performance, and lessons learned. Include details on RNG seeding and reproducibility.

9 Implementation Notes and Tips

- Geometry stepping: ensure correct handling when a sampled ℓ would cross a boundary; clip to boundary, process boundary condition, and continue residual distance if reflective.
- Tallying track length: if crossing multiple bins during a flight, decompose the segment and accumulate proportionally in each bin.
- Performance: batch particles to improve cache locality; consider SoA layout for particle states if vectorizing; avoid synchronization in inner loops.
- Debugging: start with a small number of particles and fixed seed; instrument with assertions for probability normalization and boundary handling.

10 Validation Scenarios

- Homogeneous slab with known attenuation (pure absorption) as a baseline.
- Symmetry checks with reflective boundaries.
- Convergence of relative error $\sim O(N^{-1/2})$ as particle count increases.

Bibliography and Web References

References

[1] I. Lux and L. Koblinger, *Monte Carlo Particle Transport Methods: Neutron and Photon Calculations*, CRC Press, 1991. freely available here

[2] G. Fishman, *Monte Carlo: Concepts, Algorithms, and Applications*, Springer, 1996.

[3] J. M. Hammersley and D. C. Handscomb, *Monte Carlo Methods*, Methuen, Springer, 1964.

[4] E. E. Lewis and W. F. Miller, *Computational Methods of Neutron Transport*, American Nuclear Society, 1993.

[5] P. K. Romano and B. Forget, "The OpenMC Monte Carlo Code", *Annals of Nuclear Energy*, 2013. Project: `https://openmc.org/`.

[6] M. E. O'Neill, "PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation", 2014. `https://www.pcg-random.org/`.